

CIS 6270

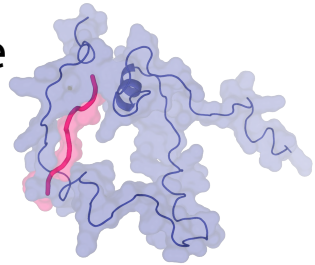
Module 3: Score Matching

Lecture 3.2: Reverse-Time Diffusion and DDPMs

Thu, Sep 17, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Before the code, let’s connect the full story

Before the code, let’s connect the full story

We start with a clean sample X_0 from the data density p_0 and gradually add noise. At time t, its noisy state is X_t.

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

Before the code, let’s connect the full story

We start with a clean sample X_0 from the data density p_0 and gradually add noise. At time t, its noisy state is X_t.

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

The state change dX_t equals the systematic drift $f(X_t, t)$ times elapsed time dt, plus the Brownian increment dW_t scaled by the noise amplitude g(t).

Before the code, let’s connect the full story

We start with a clean sample X_0 from the data density p_0 and gradually add noise. At time t, its noisy state is X_t.

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

The state change dX_t equals the systematic drift $f(X_t, t)$ times elapsed time dt, plus the Brownian increment dW_t scaled by the noise amplitude g(t).

Across many samples, these random paths form the density p_t. We choose the noising process so its final density approaches a distribution we can sample.

Brownian noise leaves a contribution at order dt

Brownian noise leaves a contribution at order dt

To follow those paths, first recall how Brownian motion changes over a short interval of length h.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

Brownian noise leaves a contribution at order dt

To follow those paths, first recall how Brownian motion changes over a short interval of length h.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

The Brownian displacement ΔW equals the square root of the interval h times a standard Gaussian vector ε, whose covariance is the identity matrix I.

Brownian noise leaves a contribution at order dt

To follow those paths, first recall how Brownian motion changes over a short interval of length h.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

The Brownian displacement ΔW equals the square root of the interval h times a standard Gaussian vector ε, whose covariance is the identity matrix I.

For h = 0.01, each coordinate has standard deviation 0.1 and mean squared displacement 0.01. Summed squared increments approach elapsed time, which we write as $(dW_t)^2 = dt$.

That surviving square gives the Itô curvature term

That surviving square gives the Itô curvature term

Now follow a smooth measurement $\varphi(x, t)$ in one coordinate. Its derivatives φ_t, φ_x, and φ_{xx} describe time change, slope, and curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right) dt + g\phi_x\, dW_t$$

That surviving square gives the Itô curvature term

Now follow a smooth measurement $\varphi(x, t)$ in one coordinate. Its derivatives φ_t, φ_x, and φ_{xx} describe time change, slope, and curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right) dt + g\phi_x\, dW_t$$

The measurement change $d\varphi$ equals its time derivative φ_t plus drift f times slope φ_x plus half the squared noise amplitude g^2 times curvature φ_{xx}, all times dt, plus $g\varphi_x$ times the Brownian increment dW_t.

That surviving square gives the Itô curvature term

Now follow a smooth measurement $\varphi(x, t)$ in one coordinate. Its derivatives φ_t, φ_x, and φ_{xx} describe time change, slope, and curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right) dt + g\phi_x\, dW_t$$

The measurement change $d\varphi$ equals its time derivative φ_t plus drift f times slope φ_x plus half the squared noise amplitude g^2 times curvature φ_{xx}, all times dt, plus $g\varphi_x$ times the Brownian increment dW_t.

Taylor expansion gives the factor one-half. Because $(dX_t)^2 = g(t)^2 dt$, the curvature term survives. For $\varphi(x) = x^2$ at zero, a move in either direction increases the measurement.

We can now average over the moving samples

We can now average over the moving samples

For a time-independent measurement $\varphi(x)$, averaging $\varphi(X_t)$ over samples is the same as integrating $\varphi(x)$ against the density $p_t(x)$.

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f \cdot \nabla\phi + \tfrac{1}{2}g^2\Delta\phi\right]$$

We can now average over the moving samples

For a time-independent measurement $\varphi(x)$, averaging $\varphi(X_t)$ over samples is the same as integrating $\varphi(x)$ against the density $p_t(x)$.

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f \cdot \nabla\phi + \tfrac{1}{2}g^2\Delta\phi\right]$$

The rate of change of the average measurement $E[\varphi(X_t)]$ equals the average of drift f dotted with the measurement gradient $\nabla\varphi$, plus half the squared noise amplitude g^2 times the measurement Laplacian $\Delta\varphi$, the sum of its coordinate curvatures.

We can now average over the moving samples

For a time-independent measurement $\varphi(x)$, averaging $\varphi(X_t)$ over samples is the same as integrating $\varphi(x)$ against the density $p_t(x)$.

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f \cdot \nabla\phi + \tfrac{1}{2}g^2\Delta\phi\right]$$

The rate of change of the average measurement $E[\varphi(X_t)]$ equals the average of drift f dotted with the measurement gradient $\nabla\varphi$, plus half the squared noise amplitude g^2 times the measurement Laplacian $\Delta\varphi$, the sum of its coordinate curvatures.

The Brownian integral has zero mean under the usual integrability conditions. We can therefore calculate the change in this average from the drift and curvature terms.

Moving the derivatives gives Fokker-Planck

Moving the derivatives gives Fokker-Planck

Write that average as a density integral, then integrate by parts. With vanishing boundary terms, the derivatives move from φ onto the density.

$$\partial_t p_t = -\nabla \cdot (f p_t) + \tfrac{1}{2} g(t)^2 \Delta p_t$$

Moving the derivatives gives Fokker-Planck

Write that average as a density integral, then integrate by parts. With vanishing boundary terms, the derivatives move from φ onto the density.

$$\partial_t p_t = -\nabla \cdot (f p_t) + \tfrac{1}{2} g(t)^2 \Delta p_t$$

The density's time derivative $\partial_t p_t$ equals the negative divergence of the drift flux $f p_t$, plus half the squared noise amplitude $g(t)^2$ times the density Laplacian Δp_t, which sums its second spatial derivatives.

Moving the derivatives gives Fokker-Planck

Write that average as a density integral, then integrate by parts. With vanishing boundary terms, the derivatives move from φ onto the density.

$$\partial_t p_t = -\nabla \cdot (f p_t) + \tfrac{1}{2} g(t)^2 \Delta p_t$$

The density's time derivative $\partial_t p_t$ equals the negative divergence of the drift flux fp_t, plus half the squared noise amplitude $g(t)^2$ times the density Laplacian Δp_t, which sums its second spatial derivatives.

The first term moves probability with the drift, and the second spreads it through noise. This equation describes the density of the same samples we followed with the SDE.

To generate samples, we need the score of that density

To generate samples, we need the score of that density

We want to reverse the noising process. The time-reversal equation requires the score at each noise level.

$$s_t(x) = \nabla_x \log p_t(x)$$

To generate samples, we need the score of that density

We want to reverse the noising process. The time-reversal equation requires the score at each noise level.

$$s_t(x) = \nabla_x \log p_t(x)$$

The noisy-data score $s_t(x)$ equals the spatial gradient ∇_x of the log density $p_t(x)$, giving the local direction and strength of increasing log probability.

To generate samples, we need the score of that density

The time-reversal result inserts this score into the drift.

To generate samples, we need the score of that density

The time-reversal result inserts this score into the drift. We now run noising time t backward, so dt is negative.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

To generate samples, we need the score of that density

The time-reversal result inserts this score into the drift. We now run noising time t backward, so dt is negative.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

The state change dX_t equals the forward drift f minus the squared noise amplitude $g(t)^2$ times the score $s_t(X_t)$, all times the negative time increment dt, plus g(t) times the reverse Brownian increment $d\bar{W}_t$.

To generate samples, we need the score of that density

The time-reversal result inserts this score into the drift. We now run noising time t backward, so dt is negative.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

The state change dX_t equals the forward drift f minus the squared noise amplitude $g(t)^2$ times the score $s_t(X_t)$, all times the negative time increment dt, plus g(t) times the reverse Brownian increment $d\bar{W}_t$.

With dt < 0, the score contribution moves samples toward higher noisy-data density.

To generate samples, we need the score of that density

The time-reversal result inserts this score into the drift. We now run noising time t backward, so dt is negative.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

The state change dX_t equals the forward drift f minus the squared noise amplitude $g(t)^2$ times the score $s_t(X_t)$, all times the negative time increment dt, plus g(t) times the reverse Brownian increment $d\overline{W}_t$.

With dt < 0, the score contribution moves samples toward higher noisy-data density.

We know f and g, but we still need to learn the unknown score s_t.

We can calculate targets from Gaussian corruption

We can calculate targets from Gaussian corruption

For our Gaussian noising path, sample clean data X_0 and an independent standard Gaussian vector ε.

We can calculate targets from Gaussian corruption

For our Gaussian noising path, sample clean data X_0 and an independent standard Gaussian vector ε. We know the schedule at every time t.

$$X_t = a_t X_0 + \sigma_t \varepsilon$$

We can calculate targets from Gaussian corruption

For our Gaussian noising path, sample clean data X_0 and an independent standard Gaussian vector ε. We know the schedule at every time t.

$$X_t = a_t X_0 + \sigma_t \varepsilon$$

The noisy sample X_t equals the clean sample X_0 times the signal-retention coefficient a_t, plus Gaussian noise ε times the noise standard deviation σ_t.

We can calculate targets from Gaussian corruption

The corruption density $q_t(x \mid x_0)$ is Gaussian, with mean $a_t x_0$ and variance σ_t^2 in each coordinate, so we can differentiate its log density.

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}$$

We can calculate targets from Gaussian corruption

The corruption density $q_t(x \mid x_0)$ is Gaussian, with mean $a_t x_0$ and variance σ_t^2 in each coordinate, so we can differentiate its log density.

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}$$

The corruption score $\nabla_x \log q_t(x \mid x_0)$ equals the displacement from noisy point x toward mean $a_t x_0$ divided by noise variance σ_t^2, which also equals negative sampled noise ε divided by noise standard deviation σ_t.

We can calculate targets from Gaussian corruption

The corruption density $q_t(x \mid x_0)$ is Gaussian, with mean $a_t x_0$ and variance σ_t^2 in each coordinate, so we can differentiate its log density.

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}$$

The corruption score $\nabla_x \log q_t(x \mid x_0)$ equals the displacement from noisy point x toward mean $a_t x_0$ divided by noise variance σ_t^2, which also equals negative sampled noise ε divided by noise standard deviation σ_t.

We know the clean sample and the noise we added during training, so we can calculate this target for every clean-and-noisy pair.

Averaging those targets recovers the unknown score

Averaging those targets recovers the unknown score

Several clean states can produce the same noisy point. Bayes' rule tells us how much weight to give each possible clean state.

$$s_t(x) = \mathbb{E}[\nabla_x \log q_t(x \mid X_0) \mid X_t = x]$$

Averaging those targets recovers the unknown score

Several clean states can produce the same noisy point. Bayes' rule tells us how much weight to give each possible clean state.

$$s_t(x) = \mathbb{E}[\nabla_x \log q_t(x \mid X_0) \mid X_t = x]$$

The noisy-data score $s_t(x)$ equals the average corruption score $\nabla_x \log q_t(x \mid X_0)$, weighting possible clean states X_0 by how likely they are to have produced the observed noisy state $X_t = x$.

Averaging those targets recovers the unknown score

Several clean states can produce the same noisy point. Bayes' rule tells us how much weight to give each possible clean state.

$$s_t(x) = \mathbb{E}[\nabla_x \log q_t(x \mid X_0) \mid X_t = x]$$

The noisy-data score $s_t(x)$ equals the average corruption score $\nabla_x \log q_t(x \mid X_0)$, weighting possible clean states X_0 by how likely they are to have produced the observed noisy state $X_t = x$.

Each clean state contributes a direction toward its own corruption mean. Their posterior-weighted average gives the score of the full noisy-data density.

We can learn that conditional average with squared error

We can learn that conditional average with squared error

The network s_θ receives only noisy state X_t and time t. Its parameters θ determine its prediction, and a positive weight w(t) controls each noise level's contribution.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t) \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

We can learn that conditional average with squared error

The network s_θ receives only noisy state X_t and time t. Its parameters θ determine its prediction, and a positive weight w(t) controls each noise level's contribution.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t) \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

The denoising loss L_{DSM} averages the weight w(t) times the squared error between score prediction $s_\theta(X_t, t)$ and target $-\varepsilon/\sigma_t$, over times t, clean samples X_0, and Gaussian noise ε.

We can learn that conditional average with squared error

The network s_θ receives only noisy state X_t and time t. Its parameters θ determine its prediction, and a positive weight w(t) controls each noise level's contribution.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t) \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

The denoising loss L_{DSM} averages the weight w(t) times the squared error between score prediction $s_\theta(X_t, t)$ and target $-\varepsilon/\sigma_t$, over times t, clean samples X_0, and Gaussian noise ε.

At the population optimum, squared error gives the conditional mean of the targets.

We can learn that conditional average with squared error

The network s_θ receives only noisy state X_t and time t. Its parameters θ determine its prediction, and a positive weight w(t) controls each noise level's contribution.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t) \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

The denoising loss L_{DSM} averages the weight w(t) times the squared error between score prediction $s_\theta(X_t, t)$ and target $-\varepsilon/\sigma_t$, over times t, clean samples X_0, and Gaussian noise ε.

At the population optimum, squared error gives the conditional mean of the targets.

With sufficient capacity and successful optimization, the network approximates the marginal score needed for reversal.

In the code, we will predict the sampled noise

In the code, we will predict the sampled noise

We can implement the same training by asking a network ε_θ, with parameters θ, to predict the Gaussian noise.

$$s_\theta(x,t) = -\frac{\varepsilon_\theta(x,t)}{\sigma_t}$$

In the code, we will predict the sampled noise

We can implement the same training by asking a network ε_θ, with parameters θ, to predict the Gaussian noise.

$$s_\theta(x, t) = -\frac{\varepsilon_\theta(x, t)}{\sigma_t}$$

The predicted score $s_\theta(x, t)$ equals negative predicted noise $\varepsilon_\theta(x, t)$ divided by the known noise standard deviation σ_t.

In the code, we will predict the sampled noise

Choose the score-loss weight w(t) = σ_t^2. Substituting the noise prediction cancels the noise-scale factors.

$$\mathcal{L} = \mathbb{E}\left[\|\varepsilon_\theta(X_t, t) - \varepsilon\|^2\right]$$

In the code, we will predict the sampled noise

Choose the score-loss weight w(t) = σ_t^2. Substituting the noise prediction cancels the noise-scale factors.

$$\mathcal{L} = \mathbb{E}\left[\|\varepsilon_\theta(X_t, t) - \varepsilon\|^2\right]$$

The loss L averages the squared distance between the network's predicted noise $\varepsilon_\theta(X_t, t)$ and the actual Gaussian draw ε, over sampled times and training examples.

In the code, we will predict the sampled noise

Choose the score-loss weight w(t) = σ_t^2. Substituting the noise prediction cancels the noise-scale factors.

$$\mathcal{L} = \mathbb{E}\left[\|\varepsilon_\theta(X_t, t) - \varepsilon\|^2\right]$$

The loss L averages the squared distance between the network's predicted noise $\varepsilon_\theta(X_t, t)$ and the actual Gaussian draw ε, over sampled times and training examples.

We will create a noisy batch, predict its noise, and minimize this loss. Then we can convert the prediction to a score and use it in the reverse process to generate samples.

Now we can train a network to predict the added noise

Now we can train a network to predict the added noise

A batch contains a clean tensor, a sampled time, and a sampled noise tensor.

Now we can train a network to predict the added noise

A batch contains a clean tensor, a sampled time, and a sampled noise tensor. The network receives the noisy tensor and time, then predicts the noise.

Now we can train a network to predict the added noise

A batch contains a clean tensor, a sampled time, and a sampled noise tensor. The network receives the noisy tensor and time, then predicts the noise.

Our next examples show how the architecture changes for vectors, images, and protein embeddings.

One training batch at time step t=50

One training batch at time step t=50

Suppose a mini-batch contains three clean data points:

$$x_0^{(1)} = 2.0, \qquad x_0^{(2)} = -1.0, \qquad x_0^{(3)} = 0.5$$

$$\bar{\alpha}_{50} = 0.8$$

One training batch at time step t=50

Suppose a mini-batch contains three clean data points:

$$x_0^{(1)} = 2.0, \qquad x_0^{(2)} = -1.0, \qquad x_0^{(3)} = 0.5$$

$$\bar{\alpha}_{50} = 0.8$$

Sample one noise value for each data point:

$$\epsilon^{(1)} = 0.5, \qquad \epsilon^{(2)} = -0.2, \qquad \epsilon^{(3)} = 1.0$$

One training batch at time step t=50

Suppose a mini-batch contains three clean data points:

$$x_0^{(1)} = 2.0, \qquad x_0^{(2)} = -1.0, \qquad x_0^{(3)} = 0.5$$

$$\bar{\alpha}_{50} = 0.8$$

Sample one noise value for each data point:

$$\epsilon^{(1)} = 0.5, \qquad \epsilon^{(2)} = -0.2, \qquad \epsilon^{(3)} = 1.0$$

Create the noisy batch:

$$x_{50}^{(i)} = \sqrt{0.8}\, x_0^{(i)} + \sqrt{0.2}\, \epsilon^{(i)}$$

One training batch at time step t=50

This gives approximately:

$$x_{50} = (2.01,\ -0.98,\ 0.89)$$

One training batch at time step t=50

This gives approximately:

$$x_{50} = (2.01,\ -0.98,\ 0.89)$$

The neural network receives the whole noisy batch and the time step:

$$\epsilon_\theta(x_{50}, 50) = (0.4,\ -0.1,\ 0.8)$$

One training batch at time step t=50

This gives approximately:

$$x_{50} = (2.01,\ -0.98,\ 0.89)$$

The neural network receives the whole noisy batch and the time step:

$$\epsilon_\theta(x_{50}, 50) = (0.4,\ -0.1,\ 0.8)$$

Compare with the true noise:

$$\mathcal{L} = \frac{1}{3}\left[(0.5 - 0.4)^2 + (-0.2 + 0.1)^2 + (1.0 - 0.8)^2\right] = 0.02$$

Choose a denoiser architecture that fits the data

This gives approximately:

$$x_{50} = (2.01,\ -0.98,\ 0.89)$$

The neural network receives the whole noisy batch and the time step:

$$\epsilon_\theta(x_{50}, 50) = (0.4,\ -0.1,\ 0.8)$$

Compare with the true noise:

$$\mathcal{L} = \frac{1}{3}\left[(0.5 - 0.4)^2 + (-0.2 + 0.1)^2 + (1.0 - 0.8)^2\right] = 0.02$$

In general, the network must be time-conditioned

In general, the network must be time-conditioned

We include the time step so the network can distinguish the noise levels.

In general, the network must be time-conditioned

We include the time step so the network can distinguish the noise levels.

First, encode the timestep t.

- Then inject the time embedding into the network layers.

If your data is small (vectors in R^{64}), you can use a simple MLP

If your data is small (vectors in R^{64}), you can use a simple MLP

- A small MLP to embed the timestep t

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # Time embedding: input is scalar t, output is a vector same size as x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),  # Embed timestep
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim)  # Project to same dim as x
        )

        # Main network: predicts noise from (x_t + time_embedding)
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim)
        )

    def forward(self, x_t, t):
        # Normalize t ∈ [0, T] to [0, 1]
        t = t.float().unsqueeze(1) / 1000

        # Compute time embedding
        time_embed = self.time_mlp(t)

        # Add time context to input and predict noise
        return self.net(x_t + time_embed)
```

If your data is small (vectors in R^{64}), you can use a simple MLP

- A small MLP to embed the timestep t
- Input: scalar t

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # Time embedding: input is scalar t, output is a vector same size as x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),  # Embed timestep
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim)  # Project to same dim as x
        )

        # Main network: predicts noise from (x_t + time_embedding)
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim)
        )

    def forward(self, x_t, t):
        # Normalize t ∈ [0, T] to [0, 1]
        t = t.float().unsqueeze(1) / 1000

        # Compute time embedding
        time_embed = self.time_mlp(t)

        # Add time context to input and predict noise
        return self.net(x_t + time_embed)
```

If your data is small (vectors in R^{64}), you can use a simple MLP

- A small MLP to embed the timestep t
- Input: scalar t
- Output: vector of shape [batch_size, data_dim]

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # Time embedding: input is scalar t, output is a vector same size as x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),  # Embed timestep
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim)  # Project to same dim as x
        )

        # Main network: predicts noise from (x_t + time_embedding)
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim)
        )

    def forward(self, x_t, t):
        # Normalize t ∈ [0, T] to [0, 1]
        t = t.float().unsqueeze(1) / 1000

        # Compute time embedding
        time_embed = self.time_mlp(t)

        # Add time context to input and predict noise
        return self.net(x_t + time_embed)
```

If your data is small (vectors in R^{64}), you can use a simple MLP

- A small MLP to embed the timestep t
- Input: scalar t
- Output: vector of shape [batch_size, data_dim]
- This lets the network adapt to how much noise was added at that time step

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # Time embedding: input is scalar t, output is a vector same size as x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),  # Embed timestep
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim)  # Project to same dim as x
        )

        # Main network: predicts noise from (x_t + time_embedding)
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim)
        )

    def forward(self, x_t, t):
        # Normalize t ∈ [0, T] to [0, 1]
        t = t.float().unsqueeze(1) / 1000

        # Compute time embedding
        time_embed = self.time_mlp(t)

        # Add time context to input and predict noise
        return self.net(x_t + time_embed)
```

If your data is small (vectors in R^{64}), you can use a simple MLP

- A basic 2-layer MLP
- Input: noisy x_t + time embedding
- Output: predicted noise

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # Time embedding: input is scalar t, output is a vector same size as x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),  # Embed timestep
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim)  # Project to same dim as x
        )

        # Main network: predicts noise from (x_t + time_embedding)
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim)
        )

    def forward(self, x_t, t):
        # Normalize t ∈ [0, T] to [0, 1]
        t = t.float().unsqueeze(1) / 1000

        # Compute time embedding
        time_embed = self.time_mlp(t)

        # Add time context to input and predict noise
        return self.net(x_t + time_embed)
```

If your data is small (vectors in R^{64}), you can use a simple MLP

- Normalize t to [0, 1]
- Predict time embedding
- Add time embedding to x_t to give time-awareness
- Output predicted noise

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPNoisePredictor(nn.Module):
    def __init__(self, data_dim, time_embed_dim=64):
        super().__init__()

        # Time embedding: input is scalar t, output is a vector same size as x
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),  # Embed timestep
            nn.ReLU(),
            nn.Linear(time_embed_dim, data_dim)  # Project to same dim as x
        )

        # Main network: predicts noise from (x_t + time_embedding)
        self.net = nn.Sequential(
            nn.Linear(data_dim, 128),
            nn.ReLU(),
            nn.Linear(128, data_dim)
        )

    def forward(self, x_t, t):
        # Normalize t ∈ [0, T] to [0, 1]
        t = t.float().unsqueeze(1) / 1000

        # Compute time embedding
        time_embed = self.time_mlp(t)

        # Add time context to input and predict noise
        return self.net(x_t + time_embed)
```

DDPM

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

In the DDPM paper for images, they use a U-Net

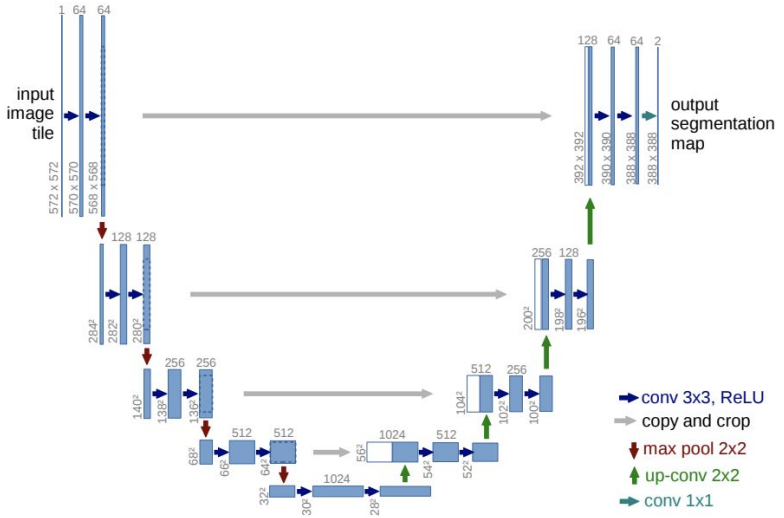


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

In the DDPM paper for images, they use a U-Net

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb

class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

First to construct the U-Net Block

- Defines 1 block in the U-Net
- Takes in a 2D image tensor with in_channels and outputs out_channels

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb

class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

First to construct the U-Net Block

- Normalize and activate input
- Convolve it
- Inject time by adding time embedding to each spatial location (broadcasting over height/width)
- Return time-aware output

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb
```

```python
class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

Next, put these blocks to make a U-Net to predict noise

- Full U-Net-like model with 3 convolutional blocks

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb

class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

Next, put these blocks to make a U-Net to predict noise

- Embeds scalar timestep t into a time embedding vector

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb

class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

Next, put these blocks to make a U-Net to predict noise

- Three stacked blocks: increase channels, then decrease
- Final 1x1 convolution maps output back to original number of channels (e.g., 1)

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb

class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

Next, put these blocks to make a U-Net to predict noise

- Compute time embedding
- Pass input through the blocks with time context
- Predict final noise image

```python
class SimpleUNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_embed_dim):
        super().__init__()
        self.norm = nn.GroupNorm(8, in_channels)
        self.activation = nn.SiLU()
        self.time_proj = nn.Linear(time_embed_dim, out_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t_emb):
        # Normalize
        h = self.norm(x)
        h = self.activation(h)
        h = self.conv(h)

        # Add broadcasted time embedding
        t_emb = self.time_proj(t_emb).unsqueeze(-1).unsqueeze(-1)
        return h + t_emb

class SimpleUNet(nn.Module):
    def __init__(self, in_channels=1, time_embed_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_embed_dim),
            nn.SiLU(),
            nn.Linear(time_embed_dim, time_embed_dim)
        )

        self.block1 = SimpleUNetBlock(in_channels, 32, time_embed_dim)
        self.block2 = SimpleUNetBlock(32, 64, time_embed_dim)
        self.block3 = SimpleUNetBlock(64, 32, time_embed_dim)
        self.out = nn.Conv2d(32, in_channels, kernel_size=1)

    def forward(self, x, t):
        # Embed t
        t = t.float().unsqueeze(1) / 1000
        t_emb = self.time_mlp(t)

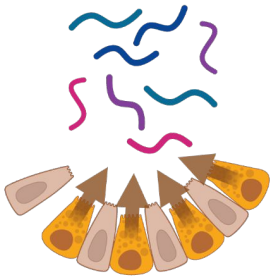
        h1 = self.block1(x, t_emb)
        h2 = self.block2(h1, t_emb)
        h3 = self.block3(h2, t_emb)
        return self.out(h3)
```

What if you want to do diffusion on embedding space?

We were actually the first ones to ever do this on protein language model embeddings!



AMP-Diffusion generates antimicrobial peptides (think antibiotics)



bioRχiv

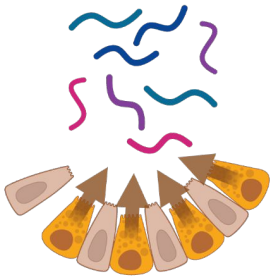
AMP-Diffusion: Integrating Latent Diffusion with Protein Language Models for Antimicrobial Peptide Generation

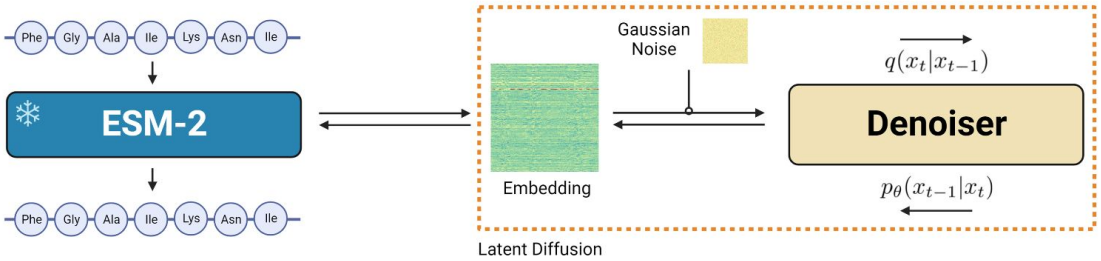
Tianlai Chen, Pranay Vure, Rishab Pulugurta, Pranam Chatterjee

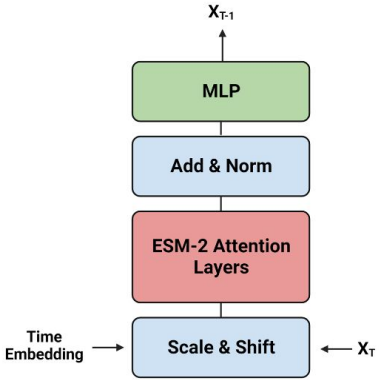
doi: https://doi.org/10.1101/2024.03.03.583201



AMP-Diffusion generates antimicrobial peptides via latent diffusion on ESM-2 embeddings



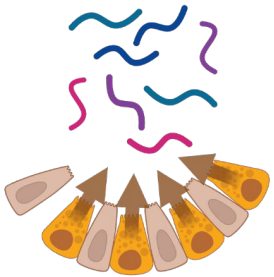




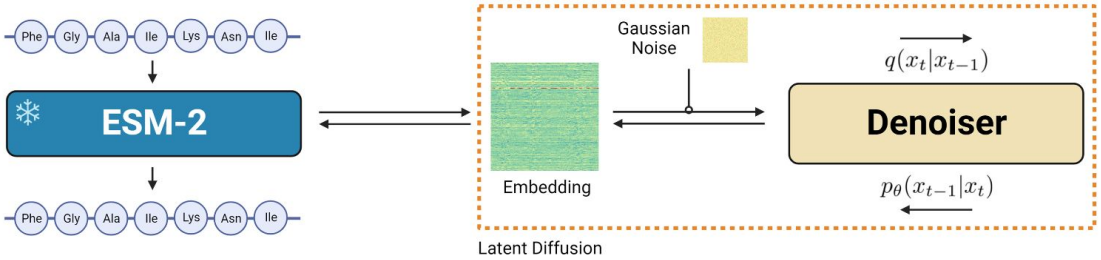
Chen,…,Chatterjee. *NeurIPS GenBio Workshop* (2023)

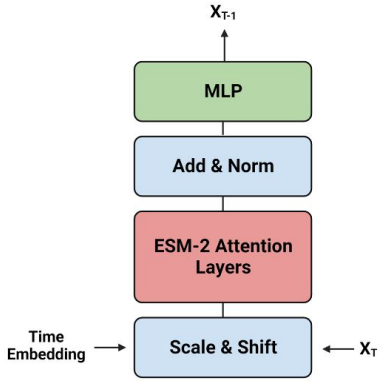


AMP-Diffusion generates antimicrobial peptides via **latent** diffusion on ESM-2 embeddings



Diffusion is applied in a compressed latent space (typically the output of a pretrained encoder) rather than in raw data space.





Chen,…,Chatterjee. *NeurIPS GenBio Workshop* (2023)



Perturb ESM-2 embeddings of AMP sequences with Gaussian noise

$$z_t = \sqrt{\alpha_t}x + \sqrt{1-\alpha_t}\epsilon \tag{1}$$

Chen,…,Chatterjee. *NeurIPS GenBio Workshop* (2023)



Perturb ESM-2 embeddings of AMP sequences with Gaussian noise via a noise scaling factor

$$z_t = \sqrt{\alpha_t}x + \sqrt{1-\alpha_t}\epsilon \tag{1}$$

$$\alpha_t = \prod_{i=1}^{t}(1-\beta_i) \tag{2}$$

Chen,…,Chatterjee. *NeurIPS GenBio Workshop* (2023)



Minimize the L2 distance between the denoised and original embeddings

$$z_t = \sqrt{\alpha_t} x + \sqrt{1 - \alpha_t} \epsilon \tag{1}$$

$$\alpha_t = \prod_{i=1}^{t} (1 - \beta_i) \tag{2}$$

$$L = \mathbb{E}_{x,t,z_t} \left[\| \hat{x}_\theta(z_t, t) - x \|_2^2 \right] \tag{3}$$

Chen,…,Chatterjee. *NeurIPS GenBio Workshop* (2023)

Let’s see it in code.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, t):
        # t: [B] → normalize → [B, 1] → [B, D]
        t = t.float().unsqueeze(1) / 1000
        return self.time_mlp(t)

class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_t, t):
        """
        z_t: [B, L, D] – noised ESM-2 embeddings
        t:   [B]       – timestep
        """
        # Time conditioning
        time_vec = self.time_embed(t).unsqueeze(1)  # [B, 1, D]
        h = z_t + time_vec  # inject time globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted x_0 or noise
```

Let’s see it in code.

- Turns scalar timestep t into a vector in R^d
- 2-layer MLP with ReLU activation
- Output can be added directly to each residue's embedding vector

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, t):
        # t: [B] → normalize → [B, 1] → [B, D]
        t = t.float().unsqueeze(1) / 1000
        return self.time_mlp(t)

class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_t, t):
        """
        z_t: [B, L, D] - noised ESM-2 embeddings
        t:   [B]       - timestep
        """
        # Time conditioning
        time_vec = self.time_embed(t).unsqueeze(1)  # [B, 1, D]
        h = z_t + time_vec  # inject time globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted x_0 or noise
```

Let’s see it in code.

- Input: Noised ESM-2 embedding

$$z_t \in \mathbb{R}^{B \times L \times D}$$

- Timestep: Scalar $t \in [0,T] \rightarrow$ mapped to R^D

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, t):
        # t: [B] → normalize → [B, 1] → [B, D]
        t = t.float().unsqueeze(1) / 1000
        return self.time_mlp(t)

class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_t, t):
        """
        z_t: [B, L, D] - noised ESM-2 embeddings
        t:   [B]       - timestep
        """
        # Time conditioning
        time_vec = self.time_embed(t).unsqueeze(1)  # [B, 1, D]
        h = z_t + time_vec  # inject time globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted x_0 or noise
```

Let’s see it in code.

- Add the time embedding to each residue — similar to biasing the sequence based on noise level

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, t):
        # t: [B] → normalize → [B, 1] → [B, D]
        t = t.float().unsqueeze(1) / 1000
        return self.time_mlp(t)

class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_t, t):
        """
        z_t: [B, L, D] – noised ESM-2 embeddings
        t:   [B]       – timestep
        """
        # Time conditioning
        time_vec = self.time_embed(t).unsqueeze(1)  # [B, 1, D]
        h = z_t + time_vec  # inject time globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted x_0 or noise
```

Let’s see it in code.

- Predicts denoised latent $\hat{x}_0$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, t):
        # t: [B] → normalize → [B, 1] → [B, D]
        t = t.float().unsqueeze(1) / 1000
        return self.time_mlp(t)

class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_t, t):
        """
        z_t: [B, L, D] - noised ESM-2 embeddings
        t:   [B]       - timestep
        """
        # Time conditioning
        time_vec = self.time_embed(t).unsqueeze(1)  # [B, 1, D]
        h = z_t + time_vec  # inject time globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted x_0 or noise
```

Training Loss = MSE

$$\mathcal{L} = \mathbb{E}_{x_0,t,\epsilon}\left[\|\hat{x}_\theta(z_t, t) - x_0\|^2\right]$$

- So we directly predict denoised latent and train using MSE loss

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class TimeEmbedding(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(1, embed_dim),
            nn.ReLU(),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, t):
        # t: [B] → normalize → [B, 1] → [B, D]
        t = t.float().unsqueeze(1) / 1000
        return self.time_mlp(t)

class AMPDiffusionDenoiser(nn.Module):
    def __init__(self, embed_dim=1280, num_layers=4, num_heads=8, ff_mult=4):
        super().__init__()

        self.time_embed = TimeEmbedding(embed_dim)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_mult * embed_dim,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.mlp = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

    def forward(self, z_t, t):
        """
        z_t: [B, L, D] - noised ESM-2 embeddings
        t:   [B]       - timestep
        """
        # Time conditioning
        time_vec = self.time_embed(t).unsqueeze(1)  # [B, 1, D]
        h = z_t + time_vec  # inject time globally

        # Pass through transformer encoder (ESM-style attention)
        h = self.transformer(h)

        # Project through MLP
        return self.mlp(h)  # predicted x_0 or noise
```

Now we can sample with Langevin dynamics

Now we can sample with Langevin dynamics

Hold the target density p fixed and use its score s = ∇ log p. Here τ measures sampler time at this one density.

$$dX_\tau = s(X_\tau)\, d\tau + \sqrt{2}\, dW_\tau$$

Now we can sample with Langevin dynamics

Hold the target density p fixed and use its score s = ∇ log p. Here τ measures sampler time at this one density.

$$dX_\tau = s(X_\tau)\, d\tau + \sqrt{2}\, dW_\tau$$

The state increment dXτ equals the score s(Xτ) times the sampler-time increment dτ, plus the square root of two times the Brownian increment dWτ.

Now we can sample with Langevin dynamics

Hold the target density p fixed and use its score s = ∇ log p. Here τ measures sampler time at this one density.

$$dX_\tau = s(X_\tau)\, d\tau + \sqrt{2}\, dW_\tau$$

The state increment dXτ equals the score s(Xτ) times the sampler-time increment dτ, plus the square root of two times the Brownian increment dWτ.

Under suitable conditions, p is stationary because the drift and random motion balance in the Fokker-Planck equation.

Next, a noise ladder helps us move between modes

Next, a noise ladder helps us move between modes

Learn scores at decreasing noise levels $\sigma_1 > \ldots > \sigma L$.

Next, a noise ladder helps us move between modes

Learn scores at decreasing noise levels $\sigma_1 > \ldots > \sigma L$. At level i, use step size η_i and index the Langevin iterations by j.

$$x^{j+1} = x^j + \eta_i s_\theta(x^j, \sigma_i) + \sqrt{2\eta_i}\, z^j,$$

$$\sigma_1 > \cdots > \sigma_L > 0$$

Next, a noise ladder helps us move between modes

Learn scores at decreasing noise levels $\sigma_1 > \ldots > \sigma L$. At level i, use step size η_i and index the Langevin iterations by j.

$$x^{j+1} = x^j + \eta_i s_\theta(x^j, \sigma_i) + \sqrt{2\eta_i}\, z^j,$$

$$\sigma_1 > \cdots > \sigma_L > 0$$

The next sample x^{j+1} equals x^j plus the step size η_i times the score at noise level σ_i, plus a Gaussian draw z^j scaled by the square root of twice η_i.

Next, a noise ladder helps us move between modes

Learn scores at decreasing noise levels $\sigma_1 > \ldots > \sigma L$. At level i, use step size η_i and index the Langevin iterations by j.

$$x^{j+1} = x^j + \eta_i s_\theta(x^j, \sigma_i) + \sqrt{2\eta_i}\, z^j,$$

$$\sigma_1 > \cdots > \sigma_L > 0$$

The next sample x^{j+1} equals x^j plus the step size η_i times the score at noise level σ_i, plus a Gaussian draw z^j scaled by the square root of twice η_i.

Large noise smooths barriers between modes.

Next, a noise ladder helps us move between modes

Learn scores at decreasing noise levels $\sigma_1 > \ldots > \sigma L$. At level i, use step size η_i and index the Langevin iterations by j.

$$x^{j+1} = x^j + \eta_i s_\theta(x^j, \sigma_i) + \sqrt{2\eta_i}\, z^j,$$

$$\sigma_1 > \cdots > \sigma_L > 0$$

The next sample x^{j+1} equals x^j plus the step size η_i times the score at noise level σ_i, plus a Gaussian draw z^j scaled by the square root of twice η_i.

Large noise smooths barriers between modes.

Smaller noise levels recover finer structure. This is annealed Langevin sampling.

Then we reverse diffusion with a score correction

Then we reverse diffusion with a score correction

Generation runs backward in noising time, so $dt < 0$.

Then we reverse diffusion with a score correction

Generation runs backward in noising time, so dt < 0. The process W_t has Brownian increments for this reversed clock.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

Then we reverse diffusion with a score correction

Generation runs backward in noising time, so dt < 0. The process W_t has Brownian increments for this reversed clock.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

The backward state increment dX_t equals the forward drift $f(X_t, t)$ minus the squared noise scale $g(t)^2$ times the score $s_t(X_t)$, all multiplied by the negative time increment dt, plus g(t) times the reverse Brownian increment $d\mathrm{W}_t$.

Then we reverse diffusion with a score correction

Generation runs backward in noising time, so dt < 0. The process W_t has Brownian increments for this reversed clock.

$$dX_t = \left[f(X_t, t) - g(t)^2 s_t(X_t) \right] dt + g(t)\, d\overline{W}_t,$$

$$dt < 0$$

The backward state increment dX_t equals the forward drift $f(X_t, t)$ minus the squared noise scale $g(t)^2$ times the score $s_t(X_t)$, all multiplied by the negative time increment dt, plus g(t) times the reverse Brownian increment $d\mathrm{W}_t$.

Because dt is negative, the score correction moves the sample toward increasing density.

Now let’s measure time from the start of generation

Now let’s measure time from the start of generation

Set $\tau = T - t$ and $Y\tau = X_t$. Now $d\tau$ is positive.

$$dY_\tau = \left[-f(Y_\tau, T-\tau) + g(T-\tau)^2 s_{T-\tau}(Y_\tau)\right] d\tau + g(T-\tau)\, d\overline{W}_\tau$$

Now let’s measure time from the start of generation

Set $\tau = T - t$ and $Y\tau = X_t$. Now $d\tau$ is positive.

$$dY_\tau = \left[-f(Y_\tau, T-\tau) + g(T-\tau)^2 s_{T-\tau}(Y_\tau)\right] d\tau + g(T-\tau)\, d\overline{W}_\tau$$

The generation increment $dY\tau$ equals negative forward drift f plus squared noise scale g^2 times the score, all evaluated at state $Y\tau$ and noising time T minus τ and multiplied by $d\tau$, plus the scaled reverse Brownian increment.

Now let’s measure time from the start of generation

Set $\tau = T - t$ and $Y\tau = X_t$. Now $d\tau$ is positive.

$$dY_\tau = \left[-f(Y_\tau, T-\tau) + g(T-\tau)^2 s_{T-\tau}(Y_\tau)\right] d\tau + g(T-\tau)\, d\overline{W}_\tau$$

The generation increment $dY\tau$ equals negative forward drift f plus squared noise scale g^2 times the score, all evaluated at state $Y\tau$ and noising time T minus τ and multiplied by $d\tau$, plus the scaled reverse Brownian increment.

We will use this positive generation clock for numerical guidance updates, so every step-size sign remains visible.

Let’s check that the reverse drift preserves a Gaussian

Let’s check that the reverse drift preserves a Gaussian

For VP diffusion at standard Gaussian equilibrium, the score is –x.

$$-f + g^2 s = \tfrac{1}{2}\beta x + \beta(-x)$$

$$= -\tfrac{1}{2}\beta x$$

Let’s check that the reverse drift preserves a Gaussian

For VP diffusion at standard Gaussian equilibrium, the score is –x.

$$-f + g^2 s = \tfrac{1}{2}\beta x + \beta(-x)$$

$$= -\tfrac{1}{2}\beta x$$

The generation drift negative f plus g^2s equals one-half βx plus β times negative x, giving the restoring drift negative one-half βx.

Let’s check that the reverse drift preserves a Gaussian

For VP diffusion at standard Gaussian equilibrium, the score is –x.

$$-f + g^2 s = \tfrac{1}{2}\beta x + \beta(-x)$$

$$= -\tfrac{1}{2}\beta x$$

The generation drift negative f plus g^2s equals one-half βx plus β times negative x, giving the restoring drift negative one-half βx.

The reverse process also preserves the standard Gaussian.

Let’s check that the reverse drift preserves a Gaussian

For VP diffusion at standard Gaussian equilibrium, the score is –x.

$$-f + g^2 s = \tfrac{1}{2}\beta x + \beta(-x)$$

$$= -\tfrac{1}{2}\beta x$$

The generation drift negative f plus g^2s equals one-half βx plus β times negative x, giving the restoring drift negative one-half βx.

The reverse process also preserves the standard Gaussian. Simply reversing the drift would make the equilibrium spread outward.

Now we can discretize the path into DDPM steps